Status	Finished
Started	Wednesday, 21 October 2026, 6:47 PM
Completed	Wednesday, 21 October 2026, 7:00 PM
Duration	13 mins 16 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

def get_line_from_file():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    try:
        with open(filename, 'r') as file:
            for current_index, line in enumerate(file, 1):
                if current_index == line_number:
                    print(line.rstrip('\n'))
                    return
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    get_line_from_file()

```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import re
from collections import Counter

def process_input():
    input_text = sys.stdin.read()
    if not input_text.strip():
        return

    words = re.findall(r'\b\w+\b', input_text.lower())
    word_counts = Counter(words)

    with open('output.txt', 'w') as file:
        for word, count in sorted(word_counts.items()):
            file.write(f"{word}: {count}\n")

if __name__ == "__main__":
    process_input()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def insert_line_at_position():
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 3:
        return

    filename = input_lines[0].strip()
    position = int(input_lines[1].strip())
    new_content = input_lines[2]

    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        lines = []

    cleaned_lines = []
    for i, line in enumerate(lines):
        if not line.endswith('\n') and i != len(lines) - 1:
            cleaned_lines.append(line + '\n')
        else:
            cleaned_lines.append(line)

    if cleaned_lines and not cleaned_lines[-1].endswith('\n'):
        cleaned_lines[-1] = cleaned_lines[-1] + '\n'

```

```
    insert_index = position - 1

    while len(cleaned_lines) < insert_index:
        cleaned_lines.append('\n')

    cleaned_lines.insert(insert_index, new_content + '\n')

    with open('output.txt', 'w') as out_file:
        out_file.writelines(cleaned_lines)

if __name__ == "__main__":
    insert_line_at_position()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
		Line one.
		Line two.
		eerht eniL.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def reverse_specific_line():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    target_line_number = int(input_data[1].strip())

    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        return

    updated_lines = []
    for current_index, line in enumerate(lines, 1):
        if current_index == target_line_number:
            has_newline = line.endswith('\n')
            content = line.rstrip('\r\n')


            if content.endswith('.'):
                reversed_content = content[:-1][::-1] + '.'
            else:
                reversed_content = content[::-1]

            if has_newline:
                reversed_content += '\n'
            updated_lines.append(reversed_content)
        else:
```

```
            updated_lines.append(line)

    with open('output.txt', 'w') as out_file:
        out_file.writelines(updated_lines)

if __name__ == "__main__":
    reverse_specific_line()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`output1.txt`
		`This is the source file.`
		`It contains multiple lines of text.`
		`Here is another line.`

Answer: (penalty regime: 0 %)

```
import sys

def copy_file_contents():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    source_filename = input_data[0].strip()
    destination_filename = input_data[1].strip()

    try:
        with open(source_filename, 'r') as source_file:
            content = source_file.read()

        with open(destination_filename, 'w') as dest_file:
            dest_file.write(content)
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    copy_file_contents()

```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`output1.txt`
			`This is the source file.`
			`It contains multiple lines of text.`
			`Here is another line.`
				`This is the source file.`
				`It contains multiple lines of text.`
				`Here is another line.`
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`output2.txt`
			`Hello, world!`
			`Python programming is amazing.`
			`Let's copy this text to another file.`
				`Hello, world!`
				`Python programming is amazing.`
				`Let's copy this text to another file.`
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input3.txt`
		`output3.txt`
			`Single line.`	`Single line.`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def replace_specific_line():
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 3:
        return

    filename = input_lines[0].strip()
    target_line_number = int(input_lines[1].strip())
    new_content = input_lines[2]

    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        lines = []

    updated_lines = []
    max_lines = max(len(lines), target_line_number)

    for i in range(max_lines):
        current_index = i + 1
        if current_index == target_line_number:
            updated_lines.append(new_content + '\n')
        elif i < len(lines):
            line = lines[i]
            if not line.endswith('\n') and current_index != max_lines:
                line += '\n'
            updated_lines.append(line)
```

```
        else:
            updated_lines.append('\n')

    if lines and not lines[-1].endswith('\n') and target_line_number != len(lines):
        if updated_lines:
            updated_lines[-1] = updated_lines[-1].rstrip('\r\n')

    with open('output.txt', 'w') as out_file:
        out_file.writelines(updated_lines)

if __name__ == "__main__":
    replace_specific_line()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
		Updated line two.
			Line one.
			Updated line two.
			Line three.
			Line four.
				Line one.
				Updated line two.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
		Line B Updated.
			Line A.
			Line B Updated.
			Line C.
				Line A.
				Line B Updated.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys

def count_total_words():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()

    try:
        with open(filename, 'r') as file:
            content = file.read()
            word_count = len(content.split())
            print(f"Total words: {word_count}")
    except FileNotFoundError:
        print("Total words: 0")

if __name__ == "__main__":
    count_total_words()

```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def delete_specific_line():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    target_line_number = int(input_data[1].strip())

    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        return

    updated_lines = []
    for current_index, line in enumerate(lines, 1):
        if current_index != target_line_number:
            updated_lines.append(line)

    with open('output.txt', 'w') as out_file:
        out_file.writelines(updated_lines)

if __name__ == "__main__":
    delete_specific_line()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import sys

def find_longest_word():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()

    try:
        with open(filename, 'r') as file:
            content = file.read()

            words = content.split()
            cleaned_words = [word.strip(".,!?;:()[]{}'\"") for word in words]

            if not cleaned_words:
                print("Longest word: ")
                return

            longest_word = max(cleaned_words, key=len)
            print(f"Longest word: {longest_word}")
    except FileNotFoundError:
        print("Longest word: ")

if __name__ == "__main__":
    find_longest_word()

```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import sys

def find_palindromes():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()

    try:
        with open(filename, 'r') as file:
            content = file.read()

        words = content.split()

        palindromes = [word for word in words if word == word[::-1]]

        with open('output.txt', 'w') as out_file:
            for word in palindromes:
                out_file.write(word + '\n')

    except FileNotFoundError:
        with open('output.txt', 'w') as out_file:
            pass

if __name__ == "__main__":
    find_palindromes()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.